

فهرست مطالب

۳	۱	مقدمه
۳	۲	معماری کلی مترجم
۳	۳	ساختار داخلی کلاس
۴	۱.۳	globalDecls
۴	۲.۳	declaredGlobals
۴	۳.۳	tryLabelStack
۴	۴.۳	continueLabelStack
۴	۵.۳	powerReplacement
۵	۶.۳	Counters
۶	۴	فرآیند کلی ترجمه
۶	۵	ترجمه ریشه برنامه
۷	۶	ترجمه دستورات
۷	۷	ترجمه تعریف متغیر
۸	۸	تبدیل نوع داده‌ها
۸	۹	تعریف متغیرهای سراسری
۸	۱۰	ترجمه عبارت‌ها
۸	۱.۱۰	تبدیل مقادیر بولی
۹	۲.۱۰	جایگزینی عبارت‌های توان
۱۰	۱۱	ترجمه ساختارهای کنترلی
۱۰	۱۲	ترجمه دستور شرط
۱۰	۱۳	ترجمه حلقه While
۱۱	۱۴	ترجمه حلقه For
۱۲	۱۵	پیاده‌سازی دستور Break
۱۲	۱۶	پیاده‌سازی دستور Continue
۱۳	۱۷	تولید بلوک‌های برنامه
۱۳	۱۸	تورفتگی خروجی
۱۴	۱۹	مدیریت استثناءها
۱۴	۲۰	متغیرهای وضعیت Exception
۱۴	۲۱	ترجمه بلوک Try
۱۵	۲۲	ترجمه دستور Throw
۱۵	۲۳	ترجمه بلوک Catch

۱۶	۲۴ بررسی تقسیم بر صفر
۱۶	۲۵ پیدا کردن تمام عملگرهای تقسیم
۱۷	۲۶ تولید بررسی قبل از تقسیم
۱۷	۲۷ بررسی عملگر /=
۱۸	۲۸ پیاده‌سازی عملگر توان
۱۸	۲۹ ایده اصلی
۱۸	۳۰ شناسایی عملگر توان
۱۹	۳۱ تولید حلقه محاسبه توان
۱۹	۳۲ جایگزینی عبارت توان
۲۰	۳۳ ترجمه عبارت‌ها
۲۰	۳۴ ترتیب اجرای عملیات
۲۱	۳۵ جمع‌بندی

۱ مقدمه

هدف این پروژه طراحی و پیاده‌سازی یک مترجم برای زبان Hash است که بتواند برنامه‌های نوشته شده در این زبان را به مدل معادل در زبان Promela تبدیل کند. خروجی تولید شده توسط این مترجم در ابزار SPIN مورد استفاده قرار می‌گیرد تا ویژگی‌های مختلف سیستم مانند، Liveness Safety و Reachability مورد بررسی قرار گیرند. در این پروژه از ANTLR برای تولید Parser و Visitor استفاده شده است. پس از تولید درخت نحوی، (AST) کلاس PromelaTranslator وظیفه پیمایش این درخت و تولید کد معادل Promela را بر عهده دارد. بر خلاف یک ترجمه صرفاً نحوی، این مترجم علاوه بر تبدیل دستورات، مسئول اضافه کردن بررسی‌های لازم برای مدل‌سازی صحیح برنامه نیز هست. مهم‌ترین این بررسی‌ها عبارت‌اند از:

- بررسی تقسیم بر صفر
 - شبیه‌سازی عملگر توان
 - مدیریت Exception
 - تولید ویژگی‌های LTL
 - تولید Labelهای مورد نیاز برای مدل‌سازی حلقه‌ها
- به همین دلیل بخش عمده منطق پروژه در کلاس PromelaTranslator پیاده‌سازی شده است.

۲ معماری کلی مترجم

معماری سیستم بر پایه الگوی طراحی Visitor بنا شده است. ANTLR پس از Parse کردن برنامه، یک درخت نحوی ایجاد می‌کند که هر گره آن متناظر با یکی از قواعد Grammar است. کلاس

PromelaTranslator

از کلاس

HashBaseVisitor<String>

ارث‌بری می‌کند. در نتیجه برای هر Rule موجود در Grammar، یک تابع

visitXXX()

در اختیار مترجم قرار می‌گیرد. هر تابع مسئول ترجمه تنها همان بخش از برنامه است. به عنوان مثال:

• visitAssignmentsStatements ترجمه دستورات انتساب

• visitIfStatements ترجمه شرط

• visitWhileStatement ترجمه حلقه While

• visitForStatement ترجمه حلقه For

• visitExceptionStatements ترجمه بلوک‌های Try/Catch

این ساختار باعث شده است توسعه مترجم بسیار ساده باشد؛ زیرا اضافه شدن هر قابلیت جدید تنها نیازمند اضافه کردن Visitor متناظر است.

۳ ساختار داخلی کلاس

در ابتدای کلاس چندین ساختمان داده تعریف شده‌اند که وظیفه مدیریت فرآیند ترجمه را بر عهده دارند.

۱.۳ globalDecls

این متغیر از نوع `StringBuilder` است. تمام متغیرهای سراسری `Promela` ابتدا داخل این متغیر ذخیره می‌شوند. در پایان ترجمه، محتوای این متغیر در ابتدای فایل خروجی قرار می‌گیرد. به عنوان نمونه اگر برنامه `Hash` شامل متغیرهای

```
۱  adad x;  
۲  adad y;  
۳  boole flag;
```

باشد، خروجی داخل `globalDecls` به صورت زیر خواهد بود.

```
۱  adad x;  
۲  adad y;  
۳  boole flag;
```

۲.۳ declaredGlobals

از آنجا که ممکن است یک متغیر چندین بار در های `Visitor` مختلف مشاهده شود، لازم است از تعریف تکراری آن جلوگیری گردد. برای این منظور از یک `HashSet` استفاده شده است. هر بار که متغیری تعریف می‌شود ابتدا بررسی می‌شود که قبلاً در این مجموعه وجود داشته باشد یا خیر. در نتیجه هیچ متغیری بیش از یک بار در فایل خروجی تولید نخواهد شد.

۳.۳ tryLabelStack

در زبان `Promela` چیزی به نام `Exception` وجود ندارد. بنابراین برای شبیه‌سازی `Try/Catch` از `Label` و دستور `goto` استفاده شده است. از آنجا که ممکن است بلوک‌های `Try` به صورت تو در تو قرار گیرند، تنها نگهداری یک `Label` کافی نیست. به همین دلیل از یک `Stack` استفاده شده است. هر بار که وارد یک بلوک `Try` می‌شویم، `Label` جدید روی `Stack` قرار می‌گیرد. پس از پایان `Try` نیز این `Label` از `Stack` حذف می‌شود. به این ترتیب همیشه بالاترین عضو `Stack` نشان‌دهنده نزدیک‌ترین بلوک `Try` فعال خواهد بود.

۴.۳ continueLabelStack

دستور `continue` نیز در `Promela` رفتار متفاوتی نسبت به زبان `Hash` دارد. برای پیاده‌سازی آن، در ابتدای هر حلقه یک `Label` مخصوص محل `Update` یا ابتدای حلقه ساخته می‌شود. سپس هنگام ترجمه دستور `Continue` تنها کافی است دستور

```
۱  goto label;
```

تولید شود. `Stack` باعث می‌شود در حلقه‌های تو در تو همیشه `Continue` به حلقه صحیح منتقل گردد.

۵.۳ powerReplacement

یکی از مهم‌ترین ساختمان داده‌های پروژه، `Map` مربوط به عملگر توان است. از آنجا که `Promela` عملگر `**` را پشتیبانی نمی‌کند، ابتدا مقدار توان داخل یک متغیر کمکی محاسبه می‌شود. سپس عبارت اصلی باید با آن متغیر جایگزین گردد. برای این منظور از ساختار

`Map<String,String>`

استفاده شده است. برای مثال رابطه زیر در این `Map` ذخیره می‌شود.

```
2**5 ---> pow_result_1
```

بعداً هنگام ترجمه عبارت‌ها، اگر چنین عبارتی مشاهده شود، مستقیماً با متغیر متناظر جایگزین خواهد شد. این مکانیزم باعث می‌شود بدون تغییر سایر بخش‌های مترجم بتوان عملگر توان را به راحتی مدل‌سازی نمود.

۶.۳ Counters

سه شمارنده مستقل نیز در کلاس وجود دارد.

- loopCounter برای تولید های‌Label یکتا در حلقه‌ها
 - tryIdx برای تولید های‌Label یکتای بلوک‌های Try
 - powerCounter برای تولید نام متغیرهای کمکی عملگر توان
- وجود این شمارنده‌ها از تداخل نام‌ها در خروجی جلوگیری می‌کند.

۴ فرآیند کلی ترجمه

تمام عملیات ترجمه از متد

`translate()`

آغاز می‌شود. ورودی این تابع، ریشه درخت نحوی تولید شده توسط ANTLR است و خروجی آن یک رشته شامل کل مدل Promela خواهد بود. در واقع این تابع تنها یک نقطه شروع برای Visitor محسوب می‌شود و کنترل را به تابع

`visitStartState()`

منتقل می‌کند. بدین ترتیب تمام عملیات ترجمه به صورت بازگشتی روی گره‌های مختلف درخت انجام می‌شود.

۵ ترجمه ریشه برنامه

متد

`visitStartState()`

اولین Visitor اجرا شده در کل پروژه است. وظیفه این متد تولید ساختار کلی فایل Promela می‌باشد. ابتدا متغیرهای سراسری مورد نیاز سیستم تعریف می‌شوند. این متغیرها عبارت‌اند از:

- `divByZero`
- `outOfBound`
- `nullPointer`
- `noPermission`
- `endReached`

این متغیرها نقش Flag را دارند و برای مدل‌سازی Exceptionها و همچنین بررسی ویژگی‌های LTL مورد استفاده قرار می‌گیرند. پس از تعریف متغیرها، تمام دستورات موجود در برنامه Hash به ترتیب پیمایش شده و Visitor متناظر با هر دستور اجرا می‌شود. خروجی هر Visitor به بدنه اصلی برنامه اضافه می‌شود. در انتها ساختار زیر تولید می‌گردد.

```
1 active proctype main() {
2
3     ...
4
5     endReached = true;
6
7     endReached_label:
8     skip;
9 }
```

وجود متغیر `endReached` امکان بررسی ویژگی Reachability را فراهم می‌کند. در صورتی که اجرای برنامه به انتها برسد مقدار این متغیر برابر True خواهد شد.

۶ ترجمه دستورات

تمام دستورات موجود در Grammar ابتدا وارد Visitor زیر می‌شوند.

visitSupportedStatements()

این تابع نقش Dispatcher را بر عهده دارد. به عبارت دیگر نوع دستور را تشخیص داده و Visitor مناسب را فراخوانی می‌کند. به عنوان نمونه:

- Assignment
- If
- While
- For
- Try
- Throw
- Continue
- Break

به‌های Visitor اختصاصی خود ارسال می‌شوند. این ساختار باعث شده است هر Visitor تنها مسئول ترجمه یک Rule از Grammar باشد.

۷ ترجمه تعریف متغیر

هنگامی که مترجم به دستور زیر برسد

```
adad x = 10;
```

تابع visitAssignmentsStatements() اجرا خواهد شد. عملیات انجام شده در این تابع به ترتیب عبارت‌اند از:

۱. تبدیل نوع داده Hash به نوع متناظر در Promela

۲. تعریف متغیر در بخش Global

۳. بررسی وجود تقسیم یا توان در عبارت

۴. ترجمه عبارت سمت راست

۵. تولید دستور انتساب

در نتیجه اگر عبارت شامل عملیات خاصی نباشد خروجی نهایی برابر خواهد بود با

```
int x;  
...  
x = 10;
```

۸ تبدیل نوع داده‌ها

متد `mapType()` مسئول تبدیل نوع‌های زبان Hash به نوع‌های Promela است. در نسخه فعلی پروژه تنها دو نوع داده پشتیبانی می‌شوند.

Promela	Hash
int	adad
bool	boole

در صورتی که نوع دیگری مشاهده شود، مترجم خطای `UnsupportedOperationException` تولید خواهد کرد.

۹ تعریف متغیرهای سراسری

تعریف متغیرها توسط متد `declareGlobal()` انجام می‌شود. در این متد ابتدا بررسی می‌شود که آیا متغیر قبلاً تعریف شده است یا خیر. در صورت جدید بودن متغیر، اعلان آن به ابتدای فایل Promela اضافه می‌شود. برای مثال

```
۱ adad x;
۲ adad y;
۳ boole flag;
```

به

```
۱ int x;
۲ int y;
۳ bool flag;
```

تبدیل خواهد شد. علاوه بر این، برای تمام متغیرهای عددی یک ویژگی `Invariant` نیز تولید می‌شود که در فایل `properties.ltl` قرار می‌گیرد. به عنوان نمونه برای متغیر `x` ویژگی زیر تولید خواهد شد.

```
۱ ltl invariant_x {
۲   [] (x >= 0)
۳ }
```

۱۰ ترجمه عبارت‌ها

یکی از مهم‌ترین بخش‌های پروژه، متد

`translateExpr()`

است.

تقریباً تمام `Visitor`ها در نهایت برای تولید عبارت سمت راست انتساب یا شرط‌ها از این تابع استفاده می‌کنند. این تابع ابتدا متن خام عبارت را از درخت نحوی استخراج می‌کند. سپس تبدیل‌های لازم روی آن انجام می‌شود. در نسخه فعلی پروژه دو نوع تبدیل صورت می‌گیرد.

۱.۱۰ تبدیل مقادیر بولی

در زبان Hash مقادیر بولی با کلمات

`dorost, ghalat`

نمایش داده می‌شوند.
اما در Promela این مقادیر به صورت

true, false

هستند. بنابراین قبل از تولید خروجی این جایگزینی انجام می‌شود.
برای مثال

```
1 flag = dorost;
```

به

```
1 flag = true;
```

تبدیل خواهد شد.

۲.۱۰ جایگزینی عبارت‌های توان

پس از تولید حلقه‌های مربوط به عملگر توان، لازم است عبارت اصلی نیز اصلاح شود. فرض کنید برنامه Hash شامل عبارت زیر باشد.

```
1 x = 2 ** 5;
```

پس از تولید حلقه محاسبه توان، مقدار نهایی داخل متغیر pow_result_1 قرار خواهد گرفت.
بنابراین اگر عبارت اصلی بدون تغییر باقی بماند، خروجی Promela همچنان شامل عملگر ** خواهد بود که توسط Promela پشتیبانی نمی‌شود.
برای جلوگیری از این مشکل، متد translateExpr() جدول powerReplacement را بررسی می‌کند.
در صورتی که عبارت

$2^{**}5$

در این جدول وجود داشته باشد، آن را با

pow_result_1

جایگزین می‌کند. در نتیجه خروجی نهایی به صورت زیر خواهد بود.

```
1 pow_result_1 = ...
```

```
2  
3 x = pow_result_1;
```

به این ترتیب بدون نیاز به وجود عملگر توان در زبان مقصد، رفتار معادل به طور کامل حفظ خواهد شد.

۱۱ ترجمه ساختارهای کنترلی

یکی از مهم‌ترین وظایف مترجم، تبدیل ساختارهای کنترلی زبان Hash به ساختارهای معادل در Promela است. در نگاه اول ممکن است تصور شود که دستورات شرطی و حلقه‌ها به صورت مستقیم قابل تبدیل هستند، اما در عمل تفاوت‌های موجود میان دو زبان باعث می‌شود مترجم علاوه بر ترجمه دستورات، ساختارهای کمکی متعددی نیز تولید کند. در این پروژه برای هر نوع ساختار کنترلی یک Visitor مجزا در نظر گرفته شده است تا منطق مربوط به هر دستور مستقل باقی بماند.

۱۲ ترجمه دستور شرط

دستور شرط در زبان Hash به صورت زیر نوشته می‌شود.

```
1  age (condition) bood {
2
3      ...
4
5  }
6  vagarna {
7
8      ...
9
10 }
```

Visitor مربوط به این بخش،

`visitIfElseStatements()`

است. ابتدا شرط موجود توسط تابع `translateExpr()` ترجمه می‌شود. سپس تمام دستورات موجود در بلوک `if` و `else` استخراج می‌شوند. از آنجا که درخت نحوی ANTLR تمام فرزندان Rule را به ترتیب نگهداری می‌کند، لازم است ابتدا مشخص شود هر دستور متعلق به کدام بلوک است. به همین دلیل تابع `splitIfChildrenSupported()` تمام فرزندان Rule را پیمایش کرده و آن‌ها را به دو لیست مجزا تقسیم می‌کند. پس از آن هر بلوک توسط `renderBlock()` به کد Promela تبدیل می‌شود. در نهایت خروجی زیر تولید خواهد شد.

```
1  if
2  :: (condition) ->
3
4      ...
5
6  :: else ->
7
8      ...
9
10 fi;
```

این ساختار معادل مستقیم دستور شرط در زبان Hash است.

۱۳ ترجمه حلقه While

در زبان Hash حلقه While به صورت زیر تعریف می‌شود.

```
1  ta(condition){
2
3      ...
4
5  }
```

اما در Promela ساختار مستقیمی برای While وجود ندارد. به همین دلیل از دستور

od ... do

استفاده شده است.

Visitor مربوط به این بخش،

visitWhileStatement()

است. در ابتدای ترجمه، یک شناسه یکتا برای حلقه تولید می‌شود. به عنوان نمونه اگر شناسه حلقه برابر ۳ باشد، های‌Label زیر ساخته می‌شوند.

```
۱ loop_start_3
۲
۳ inLoop_3
۴
۵ exitLoop_3
```

وجود این ها‌Label دو مزیت مهم دارد.

• امکان بررسی ویژگی Liveness

• پیاده‌سازی دستور Continue

پس از ترجمه شرط و بدنه حلقه، خروجی نهایی به صورت زیر خواهد بود.

```
۱ loop_start_1:
۲
۳ do
۴
۵ :: (condition) ->
۶
۷ inLoop_1:
۸
۹ ...
۱۰
۱۱ :: else -> break
۱۲
۱۳ od;
۱۴
۱۵ exitLoop_1:
۱۶
۱۷ skip;
```

وجود Label خروج از حلقه باعث می‌شود در فایل ویژگی‌های LTL بتوان بررسی کرد که آیا هر بار ورود به حلقه در نهایت منجر به خروج از آن خواهد شد یا خیر.

۱۴ ترجمه حلقه For

پیاده‌سازی حلقه For نسبت به While پیچیده‌تر است. زیرا حلقه For شامل سه بخش مستقل است.

۱. مقداردهی اولیه

۲. شرط

۳. عبارت Update

در Visitor مربوط به حلقه For ابتدا مقداره‌ی اولیه تولید می‌شود. سپس مانند حلقه While یک ساختار do...od ساخته می‌شود. تفاوت اصلی در وجود یک Label جدید با نام

loop_update_i

است. ساختار کلی خروجی به صورت زیر خواهد بود.

```
1      initialization;
2
3      loop_start_1:
4
5      do
6
7      :: (condition) ->
8
9      inLoop_1:
10
11      body
12
13      loop_update_1:
14
15      update
16
17      :: else -> break
18
19      od;
20
21      exitLoop_1:
22
23      skip;
```

این ساختار دقیقاً رفتار حلقه For را شبیه‌سازی می‌کند.

۱۵ پیاده‌سازی دستور Break

در زبان Promela دستور break به صورت مستقیم وجود دارد. به همین دلیل Visitor مربوط به دستور

shekan

تنها کافی است خروجی زیر را تولید کند.

```
1      break;
```

این دستور باعث خروج از نزدیک‌ترین حلقه فعال خواهد شد. از آنجا که ساختار do...od در Promela نیز از break پشتیبانی می‌کند، نیازی به استفاده از Label یا goto در این قسمت وجود ندارد.

۱۶ پیاده‌سازی دستور Continue

بر خلاف Break، زبان Promela دستور Continue ندارد. به همین دلیل لازم است رفتار آن به صورت دستی شبیه‌سازی شود. در هنگام ورود به هر حلقه، Label مناسب داخل

continueLabelStack

قرار می‌گیرد. در حلقه While این Label برابر ابتدای حلقه است. اما در حلقه For باید ابتدا عبارت Update اجرا شود. بنابراین Continue نباید مستقیماً به ابتدای حلقه بازگردد. به همین دلیل در حلقه For مقدار موجود در Stack برابر

loop_update_i

خواهد بود.

Visitor مربوط به دستور Continue تنها کافی است بالاترین عضو Stack را خوانده و دستور زیر را تولید کند.

```
goto loop_update_i;
```

یا در حلقه While

```
goto loop_start_i;
```

بدین ترتیب رفتار Continue دقیقاً مشابه زبان Hash خواهد بود.

۱۷ تولید بلوک‌های برنامه

تقریباً تمام‌ها Visitor برای تولید بدنه دستورات از تابع `renderBlock()` استفاده می‌کنند. این تابع یک لیست از دستورات را دریافت می‌کند. سپس Visitor مناسب برای هر دستور فراخوانی شده و خروجی آن‌ها پشت سر هم قرار می‌گیرد. در صورتی که بلوک خالی باشد، دستور

```
skip;
```

تولید می‌شود. این کار باعث می‌شود خروجی Promela همواره از نظر نحوی معتبر باقی بماند.

۱۸ تورفتگی خروجی

برای افزایش خوانایی فایل تولید شده، تمام خطوط خروجی توسط تابع `indent()` قالب‌بندی می‌شوند. این تابع به ابتدای هر خط چهار فاصله اضافه می‌کند. اگرچه این مرحله تأثیری بر عملکرد مدل ندارد، اما خوانایی خروجی را به شکل قابل توجهی افزایش می‌دهد و بررسی مدل توسط کاربر را ساده‌تر می‌کند.

۱۹ مدیریت استثناءها

یکی از تفاوت‌های اساسی میان زبان Hash و Promela، پشتیبانی از مکانیزم Exception است. زبان Hash همانند بسیاری از زبان‌های برنامه‌نویسی متداول از ساختارهای

- Try
- Catch
- Finally
- Throw

پشتیبانی می‌کند. در مقابل، زبان Promela هیچ مکانیزم داخلی برای مدیریت Exception ندارد. بنابراین لازم است این قابلیت به صورت دستی مدل‌سازی شود. در این پروژه برای پیاده‌سازی Exception از دو ایده اصلی استفاده شده است.

۱. تعریف Flag برای هر نوع Exception

۲. استفاده از Label و دستور goto برای انتقال کنترل برنامه

۲۰ متغیرهای وضعیت Exception

در ابتدای مدل Promela چهار متغیر سراسری تعریف می‌شوند.

```
۱ bool divByZero = false;
۲
۳ bool outOfBound = false;
۴
۵ bool nullPointer = false;
۶
۷ bool noPermission = false;
```

هر یک از این متغیرها نشان‌دهنده وقوع یک نوع Exception هستند. در صورت بروز خطا، مقدار متغیر متناظر برابر True خواهد شد. در ادامه بلوک Catch بر اساس همین متغیر تصمیم می‌گیرد که اجرا شود یا خیر.

۲۱ ترجمه بلوک Try

Visitor مربوط به Try عبارت است از

```
visitExceptionStatements()
```

هنگام ورود به یک بلوک Try ابتدا یک Label جدید تولید می‌شود. به عنوان نمونه

```
۱ end_try_1
```

این Label روی tryLabelStack قرار می‌گیرد. تمام های Exception ایجاد شده در این محدوده به این Label منتقل خواهند شد. در انتهای بلوک Try این Label ایجاد شده و سپس بلوک Catch تولید می‌شود. بنابراین ساختار کلی خروجی به صورت زیر خواهد بود.

```

۱      ...
۲
۳      goto end_try_1;
۴
۵      ...
۶
۷      end_try_1:
۸
۹      if
۱۰
۱۱      :: (divByZero) ->
۱۲
۱۳      ...
۱۴
۱۵      fi;

```

به این ترتیب رفتار Try/Catch بدون نیاز به پشتیبانی مستقیم Promela شبیه‌سازی می‌شود.

۲۲ ترجمه دستور Throw

Visitor مربوط به دستور Throw برابر است با

```
visitThrowsException()
```

وظیفه این Visitor فعال کردن Flag مناسب و انتقال کنترل برنامه به نزدیک‌ترین بلوک Try است. برای مثال

```
۱      bendaz SefrBood();
```

به خروجی زیر تبدیل می‌شود.

```

۱      divByZero = true;
۲
۳      goto end_try_1;

```

اگر Exception از نوع دیگری باشد، Flag متناظر با همان Exception فعال خواهد شد.

۲۳ ترجمه بلوک Catch

Visitor مربوط به Catch،

```
visitCatchClause()
```

است.
در این Visitor ابتدا نوع Exception تعیین می‌شود.
سپس نام آن به Flag متناظر تبدیل می‌شود.
برای مثال

SefrBood

به

divByZero

تبدیل می‌شود.
در ادامه بلوک زیر تولید خواهد شد.

```
1  if
2
3  :: (divByZero) ->
4
5  divByZero = false;
6
7  ...
8
9  :: else ->
10
11 skip;
12
13 fi;
```

پاک کردن مقدار Flag باعث می‌شود در ادامه اجرای برنامه همان Exception مجدداً فعال نباشد.

۲۴ بررسی تقسیم بر صفر

یکی از مهم‌ترین قابلیت‌های اضافه شده به مترجم، بررسی خودکار تقسیم بر صفر است.
در زبان Hash ممکن است برنامه‌نویس عبارت زیر را بنویسد.

```
1  adad z = x / y;
```

اگر مقدار y برابر صفر باشد، اجرای مستقیم این عبارت در Promela منجر به تولید مدل نامعتبر خواهد شد.
به همین دلیل مترجم پیش از تولید عمل تقسیم، تمام عبارت را بررسی می‌کند.

۲۵ پیدا کردن تمام عملگرهای تقسیم

این کار توسط تابع

`findDivisions()`

انجام می‌شود.
این تابع به صورت بازگشتی تمام گره‌های درخت نحوی را پیمایش می‌کند. هر زمان که به گره

`MultiplicativeExpressionContext`

برسد، تمام فرزندان آن بررسی می‌شوند. اگر یکی از عملگرها برابر
 $/$

باشد، عبارت سمت راست آن استخراج می‌شود. برای مثال عبارت

```
1  a + b / (x-1)
```

به صورت زیر تحلیل خواهد شد.

`divisor = (x-1)`

تنها مخرج ذخیره می‌شود؛ زیرا در بررسی تقسیم بر صفر تنها مقدار مخرج اهمیت دارد.

۲۶ تولید بررسی قبل از تقسیم

پس از استخراج تمام مخرج‌ها، تابع

`generateDivisionChecks()`

برای هر مخرج یک قطعه کد Promela تولید می‌کند. این کد توسط تابع `checkDivisionOnly()` ساخته می‌شود. خروجی به صورت زیر است.

```
۱  if
۲
۳  :: (y == 0) ->
۴
۵  divByZero = true;
۶
۷  goto end_try_1;
۸
۹  :: else ->
۱۰
۱۱  skip;
۱۲
۱۳ fi;
```

در نتیجه اگر مخرج صفر باشد، عمل تقسیم هرگز اجرا نخواهد شد.

۲۷ بررسی عملگر /=

برای عملگر

`/=`

صرفاً بررسی مخرج کافی نیست. زیرا پس از بررسی باید عملیات انتساب نیز انجام شود. به همین دلیل تابع جداگانه‌ای با نام `checkDevByZero()` پیاده‌سازی شده است. خروجی این تابع به صورت زیر خواهد بود.

```
۱  if
۲
۳  :: (expr == 0) ->
۴
۵  divByZero = true;
۶
۷  goto end_try_1;
۸
۹  :: else ->
۱۰
۱۱  x = x / expr;
۱۲
۱۳ fi;
```

در این حالت تنها زمانی که مخرج صفر نباشد، عملیات تقسیم انجام خواهد شد.

۲۸ پیاده‌سازی عملگر توان

یکی از مهم‌ترین قابلیت‌هایی که در مترجم پیاده‌سازی شد، پشتیبانی از عملگر توان ($**$) است. زبان Hash از عملگر توان به صورت مستقیم استفاده می‌کند.
برای مثال

```
۱ adad x = 2 ** 5;
```

در حالی که زبان Promela هیچ عملگر داخلی برای محاسبه توان ندارد. بنابراین امکان ترجمه مستقیم این عبارت وجود ندارد. به همین دلیل در این پروژه تصمیم گرفته شد عملگر توان توسط یک حلقه در Promela شبیه‌سازی شود.

۲۹ ایده اصلی

محاسبه

$$a^b$$

در واقع برابر است با ضرب کردن مقدار a به تعداد b بار. بنابراین عبارت

$$2^5$$

معادل اجرای الگوریتم زیر خواهد بود.

```
۱ result = 1;  
۲  
۳ repeat 5 times  
۴  
۵ result = result * 2;
```

به همین دلیل در خروجی Promela نیز از همین منطق استفاده شده است.

۳۰ شناسایی عملگر توان

اولین مرحله، پیدا کردن تمام عملگرهای توان در عبارت است. این وظیفه بر عهده تابع

`findPowers()`

قرار دارد. این تابع همانند بررسی تقسیم بر صفر، کل درخت نحوی را به صورت بازگشتی پیمایش می‌کند. هر زمان که به گره

`PowerExpressionContext`

برسد، بررسی می‌کند که آیا عملگر POWER وجود دارد یا خیر. در صورت وجود، دو بخش عبارت استخراج می‌شوند.

• پایه (Base)

• توان (Exponent)

برای مثال در عبارت

$$2^{**}n$$

مقادیر زیر استخراج خواهند شد.

Base = ۲

Exponent = n

این دو مقدار در یک آرایه ذخیره شده و برای مرحله بعد ارسال می‌شوند.

۳۱ تولید حلقه محاسبه توان

پس از شناسایی تمام عملگرهای توان، تابع

`generatePowerLoops()`

برای هر عبارت یک حلقه جدید تولید می‌کند. برای جلوگیری از تداخل میان چند عملگر توان، برای هر مورد دو متغیر یکتا ساخته می‌شود.

`pow_result_i` •

`pow_counter_i` •

که i شماره آن عملگر توان است. نمونه خروجی تولید شده به صورت زیر است.

```
1 pow_result_1 = 1;
2
3 pow_counter_1 = 0;
4
5 do
6
7   :: (pow_counter_1 < exponent) ->
8
9   pow_result_1 =
10  pow_result_1 * base;
11
12  pow_counter_1 =
13  pow_counter_1 + 1;
14
15  :: else -> break
16
17 od;
```

در پایان حلقه مقدار نهایی توان داخل `pow_result_1` قرار خواهد داشت.

۳۲ جایگزینی عبارت توان

پس از تولید حلقه، هنوز عبارت اصلی برنامه به شکل

```
1 x = 2 ** 5;
```

وجود دارد. اگر همین عبارت مستقیماً در خروجی نوشته شود، Promela قادر به اجرای آن نخواهد بود. به همین دلیل از ساختار داده

`powerReplacement Map<String,String>`

استفاده شده است. در این Map، عبارت اصلی توان به نام متغیر نتیجه نگاشت می‌شود. برای مثال

```
1 2**5
2
3 |
4
5 V
6
7 pow_result_1
```

در نتیجه هنگام ترجمه عبارت‌ها، هر جا رشته

`۲**۵`

دیده شود، با

`pow_result_۱`

جایگزین خواهد شد.

۳۳ ترجمه عبارت‌ها

تمام عبارت‌های برنامه توسط تابع `translateExpr()` ترجمه می‌شوند. این تابع علاوه بر تبدیل مقادیر بولی Hash به مقادیر بولی Promela، تمام نگاشت‌های موجود در `powerReplacement` را نیز اعمال می‌کند. در نتیجه اگر عبارت زیر وجود داشته باشد.

```
۱ x = 2**5;
```

پس از ترجمه به شکل زیر تبدیل خواهد شد.

```
۱ x = pow_result_1;
```

این جایگزینی باعث می‌شود هیچ عملگر ناشناخته‌ای وارد فایل Promela نشود.

۳۴ ترتیب اجرای عملیات

نکته مهم این است که حلقه محاسبه توان باید قبل از استفاده از نتیجه آن تولید شود. به همین دلیل در تابع `generateChecks()` ابتدا دو عملیات انجام می‌شود.

۱. بررسی تقسیم بر صفر

۲. تولید حلقه‌های توان

سپس عبارت اصلی برنامه ترجمه می‌شود. در نتیجه ترتیب خروجی به صورت زیر خواهد بود.

```
۱ pow_result_1 = 1;
۲
۳ pow_counter_1 = 0;
۴
۵ do
۶
۷ ...
۸
۹ od;
۱۰
۱۱ x = pow_result_1;
```

این ترتیب باعث می‌شود مقدار توان قبل از استفاده محاسبه شده باشد.

۳۵ جمع‌بندی

در این پروژه یک مترجم برای تبدیل زیرمجموعه‌ای از زبان Hash به Promela طراحی و پیاده‌سازی شد. این مترجم علاوه بر تبدیل مستقیم ساختارهای نحوی، قابلیت‌هایی را نیز اضافه می‌کند که در زبان مقصد وجود ندارند. از مهم‌ترین این قابلیت‌ها می‌توان به تشخیص خودکار تقسیم بر صفر، مدل‌سازی کامل مکانیزم Exception شبیه‌سازی عملگر توان با استفاده از حلقه، تولید ویژگی‌های LTL و ایجاد Label لازم برای بررسی خواص Liveness اشاره کرد. نتیجه نهایی، مدلی از برنامه است که علاوه بر حفظ رفتار منطقی برنامه اولیه، قابلیت بررسی رسمی توسط ابزار SPIN را نیز فراهم می‌کند. بدین ترتیب می‌توان ویژگی‌های ایمنی، دسترسی‌پذیری و صحت اجرای برنامه را به صورت خودکار مورد ارزیابی قرار داد.